

PROJECT REPORT



Project Title: Parking Management System Using (AL)

Course: Computer Architecture (CA)

Architecture Framework: Pure x64 MASM (Microsoft Macro Assembler)

Target Environment: Windows Console Framework Execution

Project Members:

Rana Haseeb Shahzad (42887)

Ahmed Bin Qasim (42841)

Date: 16th June 2026

1. Introduction

Vehicle parking systems deployed in commercial venues often suffer from computational inefficiencies, manual arithmetic overhead, and structural data vulnerabilities. This project implements an automated, console-driven **Parking Management System** developed in **x64 Microsoft Macro Assembler (MASM)**.

By operating directly within the modern x64 architecture environment rather than simulated 16-bit DOS interfaces, this system showcases practical realizations of architectural primitives. It emphasizes the direct exploitation of 64-bit general-purpose CPU registers, explicit hardware memory segmentation, stack frame alignment, tracking of systemic tracking variables, and low-level control flow structures.

2. Objectives of the Project

- **Hardware-Level Architecture Utilization:** To construct a robust, production-ready menu infrastructure leveraging architectural instruction primitives.
- **State Management via Registers:** To manipulate, evaluate, and track vehicle allocations using modern x64 computing registers (*rax*, *rbx*, *rcx*, *rdx*).
- **Automated Fiscal Calculation:** To dynamically execute automated arithmetic processing for aggregate parking fees and multi-class tracking arrays.
- **Modular Code Structure:** To implement structured procedure calls utilizing native `call` and `ret` architectural workflows to enforce clean functional isolation.
- **Sub-System Mechanics:** To gain comprehensive knowledge regarding conditional branch prediction, execution tracking, and memory access patterns within an IDE ecosystem.

3. Scope of the System

The application is engineered to establish strict software bounds to safely mirror a local physical parking space.

- **Volumetric Capacity Bounds:** Hard-capped to a maximum allocation limit of 8 concurrent vehicles.
- **Distinct Architectural Vectors:** Full support for three independent classification tiers:
 - **Rikshaw** (Allocated Entry Rate: 200 units)
 - **Car** (Allocated Entry Rate: 300 units)
 - **Bus** (Allocated Entry Rate: 400 units)
- **Volatile Memory Persistence:** Data footprints reside entirely in the volatile static `.data` memory segment allocations, eliminating disk-bound external storage latencies during structural cycles.

4. System Description

4.1 UI Menu Layout

The console operational matrix displays a rigid command menu demanding numerical key register inputs:

- `Press 1` → Allocate Slot for Rikshaw
- `Press 2` → Allocate Slot for Car
- `Press 3` → Allocate Slot for Bus
- `Press 4` → Query System Records
- `Press 5` → Flush System Memory (Delete Record)
- `Press 6` → Terminate Execution Pipeline

4.2 System Execution Flow

- **Boundary Validation Engine:** When a vehicle selection flag triggers, the system evaluates the absolute vehicle count against the architectural ceiling of 8 slots.
- **Dynamic Commit Vector:** If capacity is cleared, the compilation segment maps the specific flat-fee rate to the aggregate ledger variable and steps up the target inventory counter.
- **Overflow Handling:** If calculations expose an absolute capacity state boundary, an immediate overflow alert string is logged via standard output streams.
- **Flushing Mechanism:** Selecting the data purge routine resets every tracking component, scalar accumulator, and storage offset across the program data field down to an absolute zero state.

5. Architectural Tools and Specifications

- **Target Assembly Platform:** Microsoft Macro Assembler (MASM)
- **Register Context Engine:** x64 Native Architecture Mode
- **Integrated Workspace Environment:** Microsoft Visual Studio (Insider Build Core Tools Configuration)
- **Sub-System Target Context:** Windows Console Framework Environment (`/SUBSYSTEM:CONSOLE`)

6. Project Architectural Design

6.1 Data Segment Layout (.data)

The static unfragmented memory block contains explicit definitions for menu and telemetry status text literal arrays, a central 64-bit unsigned integer holding total fiscal collections, and independent 64-bit base variables serving as localized numeric counters tracking active allocations for Rikshaws, Cars, and Buses.

6.2 Code Segment Layout (.code)

The execution flow is segmented across cleanly separated functional components to optimize branching performance:

- `main PROC` → The execution entry pipeline initializing register alignments, trapping terminal inputs, and routing branches.
- `RikshawProc / CarProc / BusProc` → Independent structural calculation blocks managing resource checks and value accumulation.
- `DisplayRecordsProc` → Formats the internal scalar memory state for output.
- `ResetMemoryProc` → Direct memory clearance sequence that zeroes out the data segment blocks.

7. Strategic Advantages of the Architecture

- **Minimal Clock Cycles:** Eliminates deep software framework layers, executing routines with maximum instruction efficiency.
- **Deterministic Resource Profiles:** Real-time register execution guarantees a predictable memory consumption model.
- **Procedural Modular Integrity:** Complete operational isolation makes code tracing, debugging, and maintaining instruction logic straightforward.

8. Structural Limitations

- **Absolute Volatility:** System records do not persist past execution instance lifecycles due to the omission of non-volatile file handling pipelines.
- **Static Allocation Bounds:** The maximum capacity boundary is strictly bound to hardcoded memory constants.
- **Strict Console Mode Bounds:** Execution is restricted to standard textual stream interfaces with no graphical layout abstraction.

9. Future Architectural Enhancements

- Incorporating Windows file system kernel bindings to write state data directly to persistent block storage.
- Transitioning static capacity thresholds to dynamic heap memory structures to allow for real-time scale changes.
- Injecting real-time event timestamp flags via hardware clock interrupt lookups.

10. Conclusion

This Parking Management System fulfills its design objectives by serving as a high-performance demonstration of pure x64 assembly architecture. Developing the system exposed core concepts of register state tracking, memory segment alignment, and conditional execution paths. The resulting application provides an excellent foundational base for writing modular assembly programs within a modern development environment.

11. Appendix Source Code:

```

;=====
; PROJECT: PARKING MANAGEMENT SYSTEM
; COURSE: COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE (COAL)
; ARCHITECTURE: x64 NATIVE (MASM)
; ENVIRONMENT: WINDOWS CONSOLE SUBSYSTEM
;=====

; --- EXTERNAL WINDOWS C-RUNTIME FUNCTIONS ---
extern printf : PROC
extern _getch : PROC
extern exit : PROC

.data
; --- SYSTEM STORAGE VARIABLES (64-bit Unsigned Integers) ---
total_amount dq 0 ; Tracks absolute total fiscal collections
total_count dq 0 ; Tracks total active parked vehicles
rikshaw_count dq 0 ; Counter for Rikshaws
car_count dq 0 ; Counter for Cars
bus_count dq 0 ; Counter for Buses
max_capacity dq 8 ; System volumetric ceiling boundary

; --- MENU LITERALS & PRINT STRINGS ---
sys_title db "===== ", 10, 13
          db " PARKING MANAGEMENT SYSTEM ", 10, 13
          db "===== ", 10, 13, 0

menu_msg db 10, 13, "Select an option from the matrix below:", 10, 13
         db "1. Allocate Slot for Rikshaw (Fee: 200)", 10, 13
         db "2. Allocate Slot for Car (Fee: 300)", 10, 13
         db "3. Allocate Slot for Bus (Fee: 400)", 10, 13
         db "4. Query System Records (Show Status)", 10, 13
         db "5. Flush System Memory (Delete All)", 10, 13
         db "6. Terminate Execution Pipeline", 10, 13, 10, 13
         db "Enter Selection Flag: ", 0

; --- TELEMETRY STATUS MESSAGES ---
success_msg db 10, 13, ">>> SUCCESS: Vehicle successfully committed to memory allocation slot.", 10, 13, 0
overflow_msg db 10, 13, ">>> ERROR OVERFLOW: Parking Lot Is Full! Allocation Rejected.", 10, 13, 0
```

```

flush_msg    db 10, 13, ">>> SYSTEM SYSTEM PURGE: Every active record has been flushed to 0.", 10, 13, 0
exit_msg     db 10, 13, ">>> Shutting down system architecture pipeline. Exiting execution...", 10, 13, 0
invalid_msg  db 10, 13, ">>> INVALID ENTRY: Please input an operational flag between (1-6).", 10, 13, 0
pause_msg    db 10, 13, "Press any key to return back to the menu matrix...", 0

; --- SYSTEM STATUS RECORD LAYOUT ---
record_hdr   db 10, 13, "----- CURRENT SYSTEM TELEMETRY RECORDS -----", 10, 13, 0
fmt_amount   db " [-] Aggregated Fiscal Ledger Balance : %lld units", 10, 13, 0
fmt_total_v   db " [-] Absolute Number of Parked Vehicles: %lld / 8", 10, 13, 0
fmt_rikshaw   db " [-] Total Allocated Rikshaw Slots   : %lld", 10, 13, 0
fmt_car       db " [-] Total Allocated Car Slots       : %lld", 10, 13, 0
fmt_bus       db " [-] Total Allocated Bus Slots        : %lld", 10, 13, 0
record_ftr    db "-----", 10, 13, 0

.code
main PROC
; --- SHADOW SPACE ALLOCATION ---
sub rsp, 40          ; Allocate shadow space (32 bytes + 8 bytes alignment)

MenuLoop:
; Print System Title Page Header
mov rcx, offset sys_title
call printf

; Print Interactive Selection Options Menu Matrix
mov rcx, offset menu_msg
call printf

; Capture single key press input safely using _getch
call _getch          ; AL register now holds ASCII character value of pressed key

; Process conditional branch prediction routes based on user's key entry
cmp al, '1'
je RouteRikshaw
cmp al, '2'
je RouteCar
cmp al, '3'
je RouteBus
cmp al, '4'
je RouteRecords
cmp al, '5'
je RouteFlush
cmp al, '6'
je RouteExit

; Fallback condition: handle unexpected keystrokes safely
mov rcx, offset invalid_msg
call printf
call PauseScreen
jmp MenuLoop

RouteRikshaw:
call RikshawProc
jmp MenuLoop

RouteCar:
call CarProc
jmp MenuLoop

RouteBus:
call BusProc
jmp MenuLoop

RouteRecords:
call DisplayRecordsProc
jmp MenuLoop

RouteFlush:
call ResetMemoryProc
jmp MenuLoop

RouteExit:
mov rcx, offset exit_msg

```



```

call printf

; Reclaim stack frame space and terminate process execution pipeline cleanly
add rsp, 40
mov ecx, 0          ; Exit code = 0 (Success)
call exit
main ENDP

```

```

=====
; FUNCTION PROCEDURES SUB-SYSTEMS
=====

```

```

RikshawProc PROC
    sub rsp, 40

    ; Check Volumetric Space Availability Boundary Constraint
    mov rax, total_count
    cmp rax, max_capacity
    jae OverflowAlert    ; Jump if total_count >= max_capacity

    ; Perform Entry Allocation Calculations
    add total_amount, 200    ; Add entry rate fee (200) to aggregate ledger
    inc total_count          ; Step up absolute counter (+1)
    inc rikshaw_count        ; Step up active category counter (+1)

    ; Print success confirmation output string
    mov rcx, offset success_msg
    call printf
    jmp ProcDone

```

```

OverflowAlert:
    mov rcx, offset overflow_msg
    call printf

```

```

ProcDone:
    call PauseScreen
    add rsp, 40
    ret

```

```

RikshawProc ENDP

```

```

CarProc PROC
    sub rsp, 40

    mov rax, total_count
    cmp rax, max_capacity
    jae OverflowAlertCar

    add total_amount, 300    ; Add entry rate fee (300) to aggregate ledger
    inc total_count
    inc car_count

    mov rcx, offset success_msg
    call printf
    jmp CarProcDone

```

```

OverflowAlertCar:
    mov rcx, offset overflow_msg
    call printf

```

```

CarProcDone:
    call PauseScreen
    add rsp, 40
    ret

```

```

CarProc ENDP

```

```

BusProc PROC
    sub rsp, 40

    mov rax, total_count
    cmp rax, max_capacity

```

```
jae OverflowAlertBus

add total_amount, 400    ; Add entry rate fee (400) to aggregate ledger
inc total_count
inc bus_count

mov rcx, offset success_msg
call printf
jmp BusProcDone

OverflowAlertBus:
mov rcx, offset overflow_msg
call printf

BusProcDone:
call PauseScreen
add rsp, 40
ret
BusProc ENDP

DisplayRecordsProc PROC
sub rsp, 40

; Print Header Separator
mov rcx, offset record_hdr
call printf

; Print Total Accumulated Cash Balance
mov rcx, offset fmt_amount
mov rdx, total_amount    ; Pass value to second register parameter
call printf

; Print Consolidated Active Count Check
mov rcx, offset fmt_total_v
mov rdx, total_count
call printf

; Print Categorized Vehicle Sub-Counts
mov rcx, offset fmt_rikshaw
mov rdx, rikshaw_count
call printf

mov rcx, offset fmt_car
mov rdx, car_count
call printf

mov rcx, offset fmt_bus
mov rdx, bus_count
call printf

; Print Footer Separator
mov rcx, offset record_ftr
call printf

call PauseScreen
add rsp, 40
ret
DisplayRecordsProc ENDP

ResetMemoryProc PROC
sub rsp, 40

; Flush all local counting and tracking parameters down to absolute zero
mov total_amount, 0
mov total_count, 0
mov rikshaw_count, 0
mov car_count, 0
mov bus_count, 0

mov rcx, offset flush_msg
call printf
```



```
    call PauseScreen
    add rsp, 40
    ret
ResetMemoryProc ENDP

PauseScreen PROC
    sub rsp, 40

    ; Display exit tracking line text
    mov rcx, offset pause_msg
    call printf

    ; Pause CPU execution loop wait sequence for a terminal key press entry
    call _getch

    add rsp, 40
    ret
PauseScreen ENDP

END
```